Processing Time Prediction
--- Model Architecture ---

Overview

This document describes the end-to-end architecture for predicting citizen service processing time (in minutes) from appointment requests. It covers data contracts, feature engineering, preprocessing, model training, validation, ensembling, inference, persistence, and quality considerations.

Goals

- Accurate predictions with stable, production-ready preprocessing
- Time-aware validation and early stopping to prevent overfitting
- An interpretable, reproducible pipeline with robust inference on minimal inputs

Primary use cases

- Appointment scheduling: estimate service duration to optimize slots
- Staffing planning: forecast processing load per section and hour
- Performance analytics: track SLA adherence and identify bottlenecks

Data contracts

Training inputs

- bookings_train.csv: historical bookings with timestamps and outcomes
- tasks.csv: task_id → section_id mapping
- staffing_train.csv: staffing levels and operational load indicators

Keys and joins

- task_id joins bookings to tasks to obtain section_id
- Staffing features are aggregated/merged by date (and hour where applicable)

Target

- processing_minutes = (check_out_time − check_in_time) in minutes
- Negative or missing durations are dropped during cleaning

Inference inputs (production)

- task1_test_inputs.csv with columns: [date, time, task_id] (and optional row_id)
- tasks.csv is required to resolve section_id at inference

Outputs

- Predicted processing time in minutes, one value per input row

Feature engineering

Features are derived deterministically from inputs and auxiliary tables. The training pipeline and inference path use identical logic.

Time features

- year, month, day, day_of_week, is_weekend, week_of_year
- hour, minute, minute_of_day
- cyclical encodings: time_sin, time_cos, dow_sin, dow_cos, month_sin, month_cos

Booking features

- num_documents, queue_number, satisfaction_rating
- log transforms: num_documents_log1p, queue_number_log1p

Staffing features

- employees_on_duty, total_task_time_minutes, load_per_employee
- lag features at 1, 3, 7 days for key staffing metrics
- rolling means (e.g., 7-day) for smoothed context

Categorical features

- task_id, section_id (one-hot encoded; unknowns ignored safely)

Notes

- Inference avoids non-deterministic binning (e.g., qcut without fixed edges)
- Missing engineered columns default to NaN and are handled downstream by the preprocessor

Why these features

- Time signals capture recurring cyclic patterns (weekday cycles, seasonal effects)
- Booking features proxy citizen/task complexity (documents, queues)
- Staffing features represent supply-side capacity and its temporal dynamics

Data dictionary (selected)

- time_sin, time_cos: cyclical encodings of minute_of_day to preserve periodicity
- dow_sin, dow_cos: cyclical encodings of day_of_week
- load_per_employee: total_task_time_minutes / max(employees_on_duty, 1)
- num_documents_log1p: log1p transform to reduce skew and stabilize trees

Preprocessing pipeline

- Categorical: OneHotEncoder(handle_unknown="ignore", sparse_output=False)
- Numeric: SimpleImputer(strategy="mean")
- Integration: ColumnTransformer([("cat", OHE, categorical_cols), ("num", Imputer, numeric_cols)])

Rationale

- handle_unknown="ignore" guarantees stability when unseen task_id/section_id appear in production
- Numeric imputation ensures complete model matrices without data leakage

Design choices

- OneHotEncoder instead of target/impact encoding to avoid leakage and simplify inference
- Dense (sparse_output=False) to keep compatibility with model APIs and feature importance extraction
- Mean imputation is robust under non-Gaussian noise and aligns with tree-based models' tolerance to scale

Base learner (XGBoost)

- Objective: squared error regression
- n_estimators: up to 3000 with early stopping
- learning_rate: 0.03
- max_depth: 6
- subsample: 0.8, colsample_bytree: 0.8
- regularization: reg_alpha=0.1, reg_lambda=1.0
- tree_method: "hist" for speed and stability

Training contract

- The model observes preprocessed matrices only; preprocessing fit occurs on training folds
- Early stopping uses a held-out temporal validation set

Why XGBoost ?

- Non-linear, handles mixed feature types well
- Strong performance out-of-the-box with early stopping and regularization
- Interpretable via feature importance; mature ecosystem

Key hyperparameters and intent

- learning_rate=0.03: conservative steps for stable early stopping
- max_depth=6: balances bias/variance on tabular data
- subsample/colsample=0.8: decorrelate trees and reduce overfitting
- reg_alpha/reg_lambda: encourage sparsity and control complexity

How it works

- Gradient boosting builds an additive ensemble of shallow decision trees. Each tree fits the residual errors of the previous trees, reducing loss iteratively.
- histogram-based tree_method speeds up split finding by binning continuous features, enabling faster training on large tabular data.

Handling missing values

- XGBoost learns default split directions for missing entries, allowing it to route NaNs without imputation inside the trees. We still impute numerics for consistency across models and to support non-GBM learners in the ensemble.

Capturing interactions

- Trees naturally model non-linearities and interactions (e.g., task × hour effects). Depth 6 allows limited higher-order interactions while controlling complexity.

Early stopping mechanics

- With an evaluation set, training halts after N rounds (e.g., 100) without metric improvement. The best_iteration_ is retained automatically for inference.

Categorical handling

- We one-hot encode task_id and section_id upstream. handle_unknown="ignore" ensures unseen categories are ignored rather than failing during transform.

Performance notes

- tree_method="hist" and moderate depth achieve good throughput on CPU.
- colsample/subsample reduce per-iteration cost and often improve generalization.

Limitations

- One-hot increases feature dimensionality with many categories.
- Without careful validation, boosting can overfit; early stopping mitigates this.

Validation and data splits

- Chronological split: training up to a split_date; validation after
- TimeSeriesSplit used where applicable to stress-test temporal generalization

Metrics

- Root Mean Square Error (RMSE):

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

- Mean Absolute Error (MAE):

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |\hat{y}_i - y_i|$$

Why time-aware validation

- Prevents look-ahead bias present in random K-fold
- Simulates production where future appointments must be predicted from past
- Stabilizes early stopping by reflecting real drift patterns

Ensemble architecture

To reduce variance and capture complementary biases, four diverse regressors are trained with a shared preprocessor and combined via inverse-RMSE weighting.

Constituents

- XGBoostRegressor (early stopping)
- LightGBMRegressor (verbosity controlled; early stopping)
- RandomForestRegressor (bagged trees)
- Ridge (linear with L2 regularization)

Per-model training

- Fit the shared preprocessor on train, transform train/validation
- For GBMs, provide eval_set for early stopping

Weighted prediction

- Let model i have validation RMSE r_i and prediction \hat{y}_i
- Compute weights:

$$w_i = \frac{1/r_i}{\sum_j 1/r_j}$$

- Final prediction per row:

$$\hat{y} = \sum_i w_i \cdot \hat{y}_i$$

Observed validation (illustrative from this project)

- XGB RMSE ≈ 12.794, MAE ≈ 9.008
- LGB RMSE ≈ 12.785, MAE ≈ 9.014
- RF RMSE ≈ 13.059, MAE ≈ 9.387
- Ridge RMSE ≈ 12.828, MAE ≈ 8.941
- Ensemble RMSE ≈ 12.769, MAE ≈ 8.983

Why ensembling

- Reduces variance by combining diverse inductive biases (linear + trees + GBMs)
- Inverse-RMSE weighting prioritizes models with stronger validation evidence
- Robust to single-model instabilities; often improves generalization a few percent

Inference pipeline (production)

Contract

- Input: dataframe with columns [date, time, task_id]
- Output: numpy array of predicted processing_minutes (float), aligned to input rows

Flow

1. Parse date/time into a unified timestamp; extract time features deterministically
2. Map task_id → section_id using tasks.csv
3. Build missing engineering fields with safe defaults (NaN → imputed later)
4. Cast categorical features to strings and fill missing with a stable token (e.g., "MISSING")
5. Apply the fitted preprocessing + model pipeline to obtain predictions

Robustness rules

- Unknown task_id/section_id: pass through; OneHotEncoder ignores unseen categories
- Mixed dtypes in categoricals: normalize to string prior to transform
- No dynamic binning at inference; only deterministic transforms are allowed

Minimal contract example

- Required: date (YYYY-MM-DD), time (HH:MM or HH:MM:SS), task_id (string)
- Optional: row_id for traceability; will be echoed back in submissions

Service expectations

- Latency: O(ms) per row on CPU; O(minutes) for batch of ~50k rows
- Throughput: vectorized preprocessing supports large batches
- Determinism: same inputs → same outputs (seeded, no random elements at inference)

Common pitfalls and mitigations

- Mixed types in categoricals → always cast to string pre-transform
- Unseen categories → ignored by OHE; prediction relies on learned priors
- Missing staffing context at inference → engineered columns left NaN → imputed

Persistence and reproducibility

- The full pipeline (preprocessor + model) is serialized via joblib to task1_xgb_model.pkl
- Version control: model artifacts and submissions tracked separately from data
- Reproducibility: RNG_SEED fixed; deterministic preprocessing ensures stable outputs

Environment notes

- Python 3.13 (Windows)
- Core libs: pandas, numpy, scikit-learn, xgboost, lightgbm, matplotlib/seaborn (EDA)

Reproducibility checklist

- Pin seeds (numpy/XGB); avoid nondeterministic transforms at inference
- Persist the full Pipeline object, not raw estimators
- Keep training/validation split logic in code, not ad-hoc notebooks only

Interpretability and diagnostics

- Feature importance extracted from the XGBoost estimator using preprocessed feature names
- Category roll-ups highlight the dominance of task/section identity and temporal patterns
- Section-level performance dashboards help identify drift or data imbalance

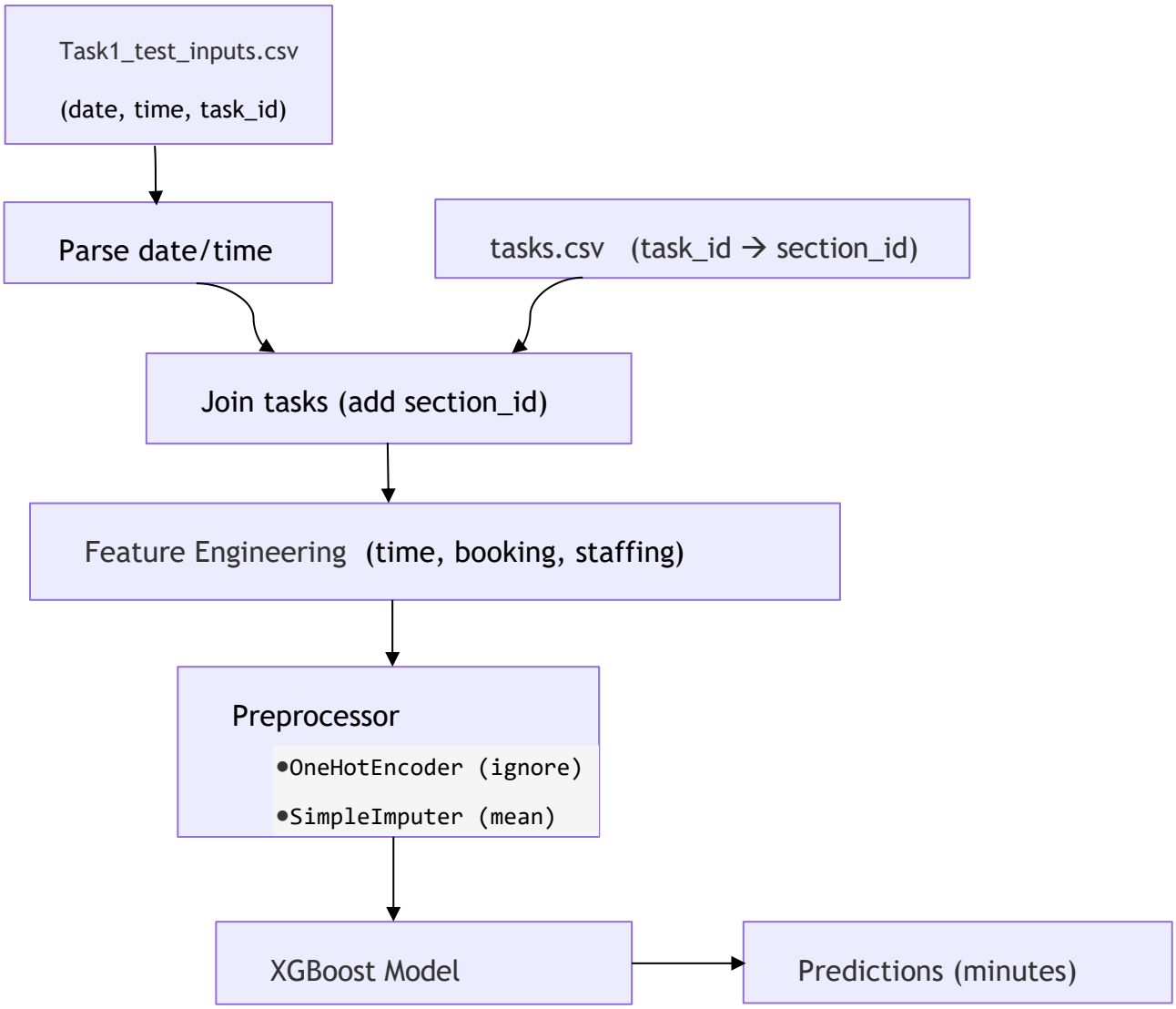
How to read importances

- High importance for task/section indicates strong dependence on service type
- Time features ranking informs scheduling sensitivity (peaks vs. off-peaks)
- Low staffing importance may indicate limited variability in historical data

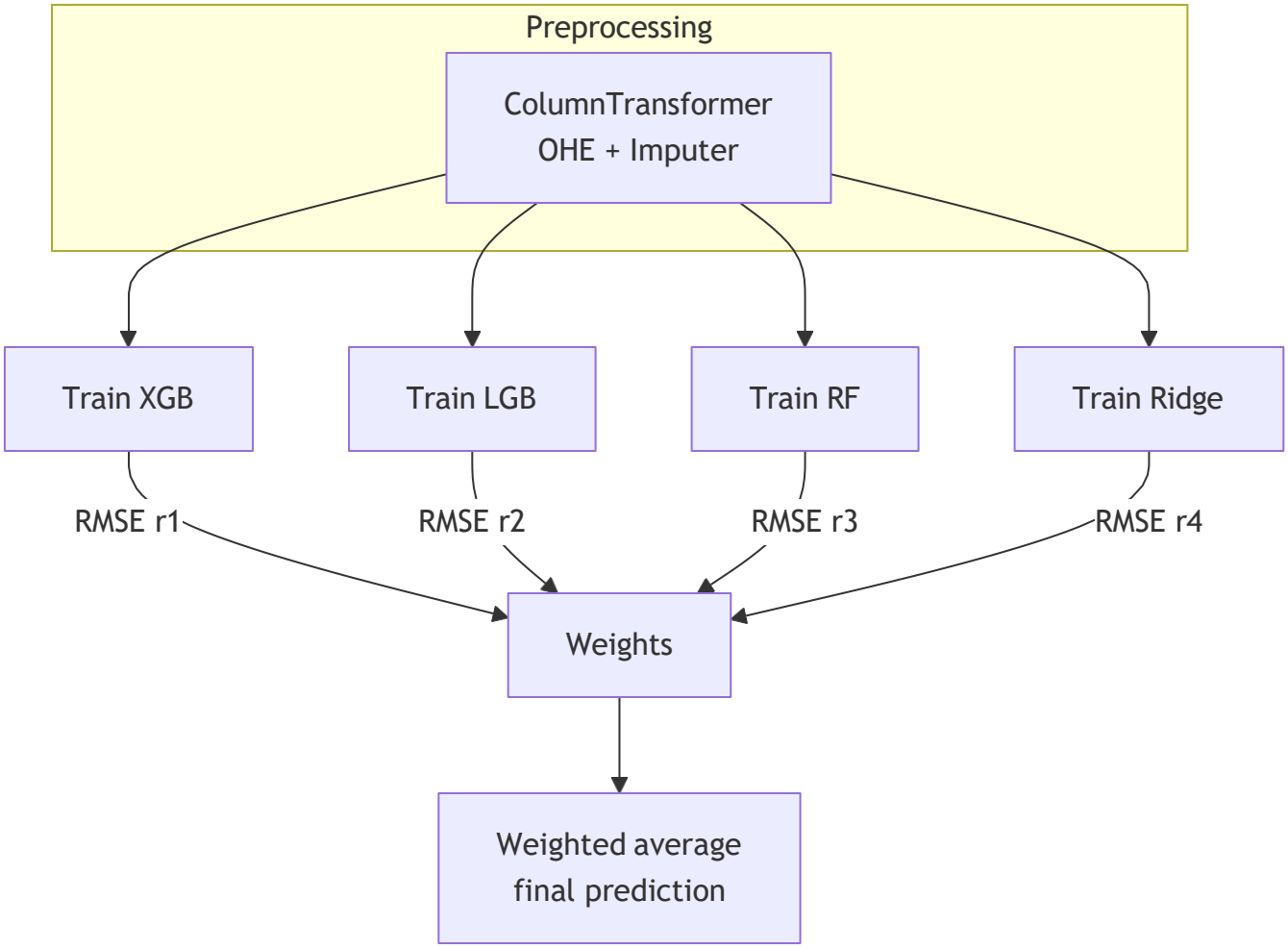
Results summary

- Baseline XGBoost validation: RMSE ≈ 13.477, MAE ≈ 9.356
- Final ensemble validation: RMSE ≈ 12.769, MAE ≈ 8.983
- Final test submission produced end-to-end using the fitted pipeline, with sensible distributional checks on outputs

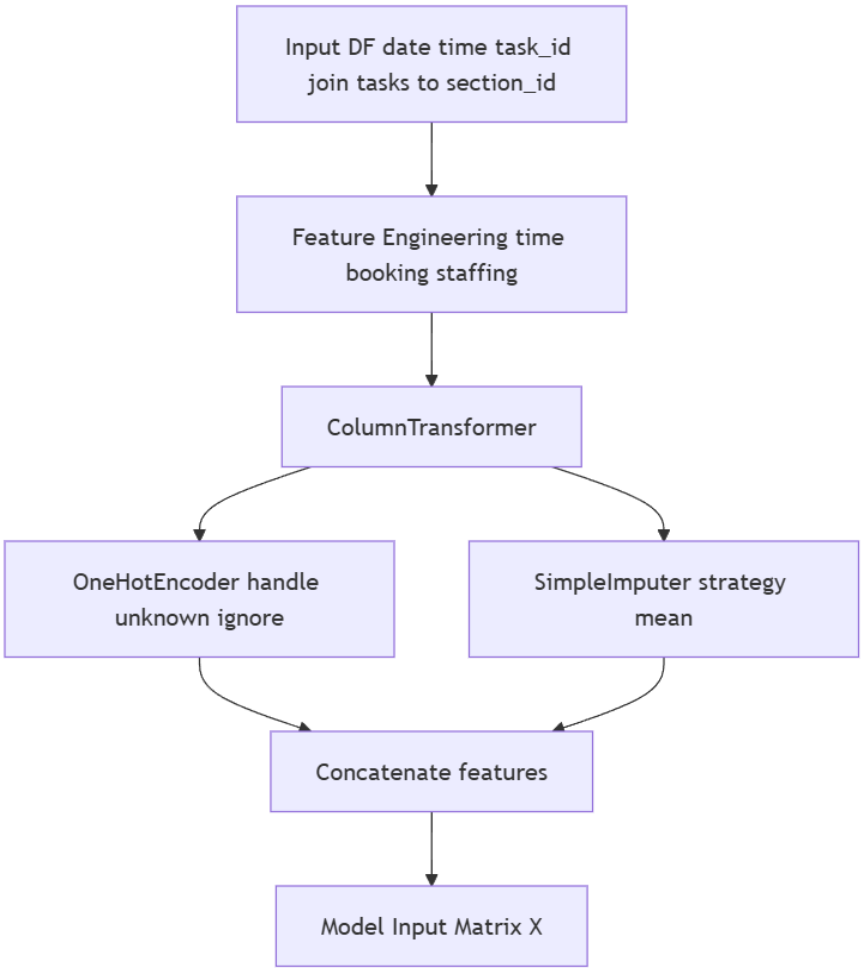
End-to-end pipeline



Mermaid – training and ensemble



Preprocessing pipeline



Model stack and blend